

Capítulo 15

C++: um C melhor — Introdução à Tecnologia de Objeto

Exercícios — Resolução Didática com Código C++

“Neste documento resolvemos os Exercícios 15.5 a 15.10 do Capítulo 15. Cada solução apresenta o programa em C++ completo, comentado, seguido de análise da arquitetura adotada, exemplos de execução real e observações didáticas. Todos os códigos foram compilados com `g++ -Wall -Wextra -std=c++14` e executados sem warnings. Esta é a primeira incursão do leitor em código C++ propriamente dito — ainda na vertente procedural da linguagem (referências, sobrecarga, argumentos default, templates, **inline**, escopo); as classes virão a partir do Capítulo 16.”

1 Exercício 15.5 — Área de um Círculo com Função Inline

Enunciado. Escreva um programa em C++ que peça o raio de um círculo e chame a função `inline circleArea` para calcular sua área.

Análise do problema

É o irmão bidimensional do Exercício 15.4. A fórmula da área do círculo é $A = \pi r^2$. Como antes, declaramos `circleArea` como **inline**: como o corpo é uma fórmula de uma linha, a expansão em linha (*inlining*) tem alto potencial de eliminação do custo de chamada de função.

Código em C++

```
1 // Exercício 15.5 - Area de um circulo com funcao inline
2 #include <iostream>
3 #include <cmath>
4
5 using namespace std;
6
7 const double PI = 3.14159;
8
9 // Funcao inline que calcula a area de um circulo
10 inline double circleArea(const double radius) {
11     return PI * pow(radius, 2);
12 }
13
14 int main() {
15     double r;
16
17     cout << "Digite o raio do circulo: ";
18     cin >> r;
19
20     cout << "A area do circulo com raio " << r
```

```

21     << " e " << circleArea(r) << endl;
22
23     return 0;
24 }

```

Execução

Saída da execução

```

$ g++ -Wall -Wextra -std=c++14 ex15_05.cpp -o ex15_05
$ ./ex15_05
Digite o raio do circulo: 3
A area do circulo com raio 3 e 28.2743

```

Discussão

Conferindo: $A = \pi \cdot 3^2 = 9\pi \approx 28,27$. Bate.

- O parâmetro `const double radius` indica que `circleArea` *não* modificará o raio recebido. Adicionar `const` a parâmetros que você não pretende alterar é uma forma de documentação executável: o compilador valida que você não quebrou essa promessa.
- Como o parâmetro é passado *por valor*, o `const` aqui é meramente uma autodisciplina interna à função — o chamador não é afetado.
- Outro ponto: `pow(radius, 2)` foi usado por simetria com 15.4, mas `radius * radius` seria mais rápido (uma multiplicação versus uma chamada de função genérica).

Dica de Desempenho

`pow(x, 2)` é um cálculo genérico que aceita expoentes arbitrários em ponto flutuante — internamente usa `exp(2 * log(x))` ou um caso especial. Para potências inteiras pequenas e conhecidas em tempo de compilação, prefira a multiplicação explícita. Em código de alto desempenho, essa diferença pode somar *ordens de magnitude*.

2 Exercício 15.6 — `tripleByValue` vs `tripleByReference`

Enunciado. Escreva um programa em C++ com duas funções alternativas, cada uma simplesmente triplicando a variável `count` definida em `main`, e compare-as:

- `tripleByValue`, que passa uma cópia de `count` por valor, triplica a cópia e retorna o novo valor;
- `tripleByReference`, que passa `count` por referência e triplica o valor original através de seu *alias*.

Análise do problema

Este é o exercício-chave para fixar a diferença entre os dois mecanismos. A pergunta não é “qual triplica?” (ambos triplicam o valor 10 e produzem 30); é “qual modifica o original?”. Vamos verificar empiricamente, imprimindo o valor de `count` *antes* e *depois* de cada chamada.

Código em C++

```

1 // Exercício 15.6 - Comparacao entre passagem por valor e passagem por referencia
2 #include <iostream>
3 using namespace std;
4
5 // (a) Passagem por valor: recebe copia, triplica e retorna o novo valor
6 int tripleByValue(int x) {
7     x *= 3;
8     return x;
9 }
10
11 // (b) Passagem por referencia: triplica o valor original atraves do alias
12 void tripleByReference(int &x) {
13     x *= 3;
14 }
15
16 int main() {
17     int count = 10;
18
19     cout << "Antes de chamar tripleByValue:      count = " << count << endl;
20     int resultado = tripleByValue(count);
21     cout << "Apos chamar tripleByValue:          count = " << count
22          << " (retorno = " << resultado << ")" << endl;
23     cout << " --> Observe que count NAO mudou.\n" << endl;
24
25     count = 10; // reinicia para a segunda demonstracao
26     cout << "Antes de chamar tripleByReference: count = " << count << endl;
27     tripleByReference(count);
28     cout << "Apos chamar tripleByReference:      count = " << count << endl;
29     cout << " --> Observe que count FOI modificado." << endl;
30
31     return 0;
32 }

```

Execução

Saída da execução

```

$ ./ex15_06
Antes de chamar tripleByValue:      count = 10
Apos chamar tripleByValue:          count = 10 (retorno = 30)
--> Observe que count NAO mudou.

Antes de chamar tripleByReference: count = 10
Apos chamar tripleByReference:      count = 30
--> Observe que count FOI modificado.

```

Discussão — Comparação das duas técnicas

Aspecto	Por valor (<code>tripleByValue</code>)	Por referência (<code>tripleByReference</code>)
Declaração	<code>int tripleByValue(int x)</code>	<code>void tripleByReference(int &x)</code>
Chamada	<code>r = tripleByValue(count);</code>	<code>tripleByReference(count);</code>
A variável original muda?	Não, <code>x</code> é cópia	Sim, <code>x</code> é <i>alias</i>
Necessita retornar valor?	Sim (senão perde-se o resultado)	Não (modifica direto)
Segurança	Melhor (não há <i>aliasing</i>)	Exige cuidado
Custo	Custo de cópia (irrelevante p/ <code>int</code>)	Sem cópia

Observação de Engenharia de Software

A passagem por valor *preserva* o argumento original, o que é desejável sempre que possível: o código torna-se mais fácil de raciocinar, porque a chamada não tem *efeitos colaterais* sobre as variáveis do chamador. A passagem por referência “ganha tempo” (sem cópia) e “ganha expressividade” (não precisa retornar via `return`), mas em troca o leitor do código precisa lembrar que, após a chamada, a variável pode ter mudado.

Boa Prática de Programação

Quando você lê código C++, o operador `&` no protótipo do parâmetro é um *sinal visível* de que aquele argumento pode ser modificado. Costume-se a procurar esse sinal — ele responde rapidamente à pergunta “esta chamada vai mexer em alguma das minhas variáveis?”.

3 Exercício 15.7 — Finalidade do Operador Unário `::`

Enunciado. Qual é a finalidade do operador unário de resolução de escopo?

Resposta

O **operador unário de resolução de escopo** (`::`) permite acessar uma variável *global* mesmo quando ela está sendo “ocultada” por uma variável *local* de mesmo nome no escopo atual. Sem o operador, qualquer referência ao identificador resolveria para a variável local (a mais próxima); com o prefixo `::`, o compilador faz a busca a partir do escopo *global*.

Demonstração

```

1 #include <iostream>
2 using namespace std;
3
4 int valor = 100;           // global

```

```

5
6 int main() {
7     int valor = 7;           // local; oculta a global
8     cout << "Local: " << valor << endl;    // imprime 7
9     cout << "Global: " << ::valor << endl;  // imprime 100
10    return 0;
11 }

```

Discussão adicional

Há ainda uma *segunda* forma do operador `::`, não coberta por esta pergunta específica mas onipresente em C++: o operador **binário** de resolução de escopo, usado para qualificar nomes pertencentes a *classes* ou namespaces. Por exemplo, `std::cout` significa “o identificador `cout` dentro do *namespace* `std`”. Ambas as formas usam o mesmo símbolo `::`, mas conceitualmente são distintas: a unária sobe até o escopo global; a binária desce até um escopo nomeado.

Observação de Engenharia de Software

A existência do operador `::` unário é um *sintoma* de um problema, não a *solução* ideal. Boas práticas de engenharia de *software* recomendam evitar nomes globais (use *namespaces*, *classes*, ou parâmetros) e, quando os globais são inevitáveis, dar-lhes nomes distintos dos locais. Em código legado ou bibliotecas não-modulares, contudo, `::` ainda salva vidas.

4 Exercício 15.8 — Template de Função min

Enunciado. Escreva um programa que use um template de função chamado `min` para determinar o menor de dois argumentos. Teste com inteiros, caracteres e ponto flutuante.

Análise do problema

A idéia da programação genérica: escrever *uma só* função que sirva para todos os tipos comparáveis. Para que isso seja possível, o corpo da função só pode usar operações que existam para todos os tipos esperados. No nosso caso, basta o operador `<`, que está definido para inteiros, ponto flutuante, caracteres e mesmo strings.

Observação de Engenharia de Software

Renomeamos a função para `minimo` no código, evitando colisão com a função `std::min` da biblioteca padrão (que está sempre disponível quando se faz `using namespace std`). É uma questão puramente didática — em código de produção, simplesmente usaríamos `std::min`.

Código em C++

```

1 // Exercício 15.8 - Template de funcao "min"
2 #include <iostream>
3 using namespace std;
4
5 // Template de funcao para retornar o menor de dois valores
6 template <typename T>
7 T minimo(T a, T b) {
8     return (a < b) ? a : b;
9 }
10
11 int main() {

```

```

12 // Teste com inteiros
13 int i1 = 47, i2 = 23;
14 cout << "min(" << i1 << ", " << i2 << ") = " << minimo(i1, i2) << endl;
15
16 // Teste com numeros de ponto flutuante
17 double d1 = 3.14, d2 = 2.71;
18 cout << "min(" << d1 << ", " << d2 << ") = " << minimo(d1, d2) << endl;
19
20 // Teste com caracteres
21 char c1 = 'M', c2 = 'A';
22 cout << "min(" << c1 << ", " << c2 << ") = " << minimo(c1, c2)
23 << " " << endl;
24
25 return 0;
26 }

```

Execução

Saída da execução

```

$ ./ex15_08
min(47, 23) = 23
min(3.14, 2.71) = 2.71
min('M', 'A') = 'A'

```

Discussão

- Para cada chamada, o compilador examina os tipos dos argumentos e *instancia* uma versão específica do template. Internamente, gera-se algo equivalente a três funções: `minimo<int>`, `minimo<double>`, `minimo<char>`.
- A comparação de caracteres usa a ordem da tabela ASCII: 'A' (65) é menor que 'M' (77), por isso o resultado é 'A'.
- Os parâmetros foram passados por valor. Se quiséssemos evitar cópias para tipos grandes, escreveríamos `const T &a`, `const T &b`.

Boa Prática de Programação

A própria biblioteca padrão oferece `std::min(a, b)` em `<algorithm>`. Em código real, prefira-a. O exercício nos pede a *implementação* didática, não a substituição da biblioteca padrão.

Erro Comum de Programação

Aplicar templates a tipos que não suportam todas as operações usadas no corpo gera *mensagens de erro de compilação extensas e difíceis de ler*. Por exemplo, `minimo(structA, structB)`, se `structA` não definir `operator<`, produzirá um erro com dezenas de linhas. C++20 oferece *concepts* para tornar essas mensagens muito mais claras.

5 Exercício 15.9 — Template de Função max

Enunciado. Escreva um programa que use um template de função chamado `max` para determinar o maior de dois argumentos. Teste com inteiros, caracteres e ponto flutuante.

Análise do problema

A solução é o “espelho” do exercício anterior: substituímos o operador `<` por `>`.

Código em C++

```

1  // Exercício 15.9 - Template de funcao "max"
2  #include <iostream>
3  using namespace std;
4
5  // Template de funcao para retornar o maior de dois valores
6  template <typename T>
7  T maximo(T a, T b) {
8      return (a > b) ? a : b;
9  }
10
11 int main() {
12     // Teste com inteiros
13     int i1 = 47, i2 = 23;
14     cout << "max(" << i1 << ", " << i2 << ") = " << maximo(i1, i2) << endl;
15
16     // Teste com numeros de ponto flutuante
17     double d1 = 3.14, d2 = 2.71;
18     cout << "max(" << d1 << ", " << d2 << ") = " << maximo(d1, d2) << endl;
19
20     // Teste com caracteres
21     char c1 = 'M', c2 = 'A';
22     cout << "max('" << c1 << ", '" << c2 << "') = '" << maximo(c1, c2)
23         << "'" << endl;
24
25     return 0;
26 }

```

Execução

Saída da execução

```

$ ./ex15_09
max(47, 23) = 47
max(3.14, 2.71) = 3.14
max('M', 'A') = 'M'

```

Discussão

A simetria com 15.8 ilustra como templates removem a duplicação de código no eixo do *tipo*, mas ainda exigem código separado quando muda a *lógica*. Se quiséssemos parametrizar também a operação de comparação, teríamos que passar uma função como argumento — conceito visto em C com *ponteiros para função* (Cap. 7), e que em C++ ganha forma elegante com *functors* e expressões *lambda* (Caps. 20 e seguintes).

Observação de Engenharia de Software

A própria biblioteca padrão implementa `std::max` e `std::min` usando templates muito parecidos com os nossos — só que com sobrecargas adicionais para iterators, listas de inicialização e funções de comparação personalizadas. Isso é uma observação consoladora: o que você escreveu aqui não é um esboço imaturo; é essencialmente o *núcleo* da implementação real.

6 Exercício 15.10 — Caça aos Erros

Enunciado. Determine se os segmentos de programa a seguir contêm erros. Explique como cada erro pode ser corrigido. Para um segmento em particular, é possível que nenhum erro esteja presente.

(a) Template cujo parâmetro de tipo não é usado

Segmento original:

```
template < class A >
int sum( int num1, int num2, int num3 )
{
    return num1 + num2 + num3;
}
```

Diagnóstico. O segmento *compila*, mas contém uma incoerência conceitual: o parâmetro de tipo `A` é declarado, porém *nunca usado* no corpo da função — todos os parâmetros e o retorno são `int` fixos. Isso anula completamente a vantagem do template: a função *não* é genérica; ela aceita apenas três `ints`. Para invocar essa função, o programador ainda precisaria escrever `sum<algumTipo>(1,2,3)`, o que é inútil.

Correção. Há duas opções:

Opção 1 — Tornar a função genuinamente genérica:

```
template <typename A>
A sum(A num1, A num2, A num3) {
    return num1 + num2 + num3;
}
```

Opção 2 — Remover o template:

```
int sum(int num1, int num2, int num3) {
    return num1 + num2 + num3;
}
```

A opção 1 é preferível se a intenção original era de fato uma soma genérica.

(b) Função void que retorna valor

Segmento original:

```
void printResults( int x, int y )
{
    cout << "A soma e " << x + y << '\n';
    return x + y;
}
```

Diagnóstico. Erro de compilação. A função foi declarada como `void` (não retorna nada), mas o corpo contém `return x + y;` — tenta-se retornar um `int`. O compilador rejeita essa contradição.

Correção. Decida o que a função deve fazer:

Opção 1 — Se for para retornar a soma, mude o tipo de retorno:

```
int printResults(int x, int y) {
    cout << "A soma e " << x + y << '\n';
    return x + y;
}
```

Opção 2 — Se for só para imprimir, remova o `return`:

```
void printResults(int x, int y) {
    cout << "A soma e " << x + y << '\n';
}
```

(c) Template sem class ou typename

Segmento original:

```
template < A >
A product( A num1, A num2, A num3 )
{
    return num1 * num2 * num3;
}
```

Diagnóstico. Erro de sintaxe. Na declaração de um parâmetro de tipo de template, deve-se usar a palavra-chave **class** ou **typename** *antes* do identificador. Sem ela, o compilador interpreta A como o nome de uma variável de algum tipo (e não acha esse tipo) e protesta.

Correção. Adicione class ou typename:

```
template <class A>
A product(A num1, A num2, A num3) {
    return num1 * num2 * num3;
}
```

ou, equivalentemente, com typename:

```
template <typename A>
A product(A num1, A num2, A num3) {
    return num1 * num2 * num3;
}
```

Observação de Engenharia de Software

class e **typename**, no contexto de parâmetros de template, são *intercambiáveis*. Historicamente, o C++ original usava apenas **class** (o que era confuso porque o parâmetro não precisa ser uma classe — pode ser qualquer tipo, inclusive primitivos). **typename** foi adicionado posteriormente para tornar a intenção clara, e é hoje a forma preferida. Mas **class** continua válida por razões de compatibilidade.

(d) Sobrecarga apenas pelo tipo de retorno

Segmento original:

```
double cube( int );
int cube( int );
```

Diagnóstico. Erro de compilação. Estas duas declarações tentam *sobrecarregar* **cube**, mas as duas versões diferem *apenas no tipo de retorno*, mantendo idênticas as listas de parâmetros (**int**). A regra de C++ é clara: a sobrecarga distingue funções pelas *assinaturas*, e a assinatura é composta pelo *nome* e pelos *tipos dos parâmetros* — *não* pelo tipo de retorno. Logo, do ponto de vista do compilador, essas duas funções são *a mesma*, e ele rejeita a duplicidade.

Por que essa regra? Imagine que o compilador encontre **int x = cube(5);**. Qual das duas escolher? A pelo tipo de retorno depende do contexto da *chamada*, e isso introduz ambiguidade insolúvel em muitos cenários. Para evitar, o C++ proibiu de saída.

Correção. Algumas opções:

Opção 1 — Dê nomes distintos:

```
double cubeD(int);    // versao em ponto flutuante
int    cubeI(int);    // versao inteira
```

Opção 2 — Diferencie pelos tipos de parâmetros:

```
double cube(double); // recebe double, retorna double
int    cube(int);    // recebe int,    retorna int
```

Opção 3 — Use um template:

```
template <typename T>
T cube(T x) { return x * x * x; }
```

Programa de verificação

Compilamos um programa contendo as quatro versões corrigidas para confirmar que todas funcionam:

```
1 // Exercício 15.10 - Versoes CORRIGIDAS dos quatro segmentos com erro
2 // (Demonstracao de que as correcoes propostas compilam e funcionam)
3
4 #include <iostream>
5 using namespace std;
6
7 // ----- (a) CORRIGIDO -----
8 // Erro original: o parametro de tipo "class A" do template nao era usado.
9 // Correcao: ou remover o template, ou substituir int por A nos parametros.
10 template <typename A>
11 A sum(A num1, A num2, A num3) {
12     return num1 + num2 + num3;
13 }
14
15 // ----- (b) CORRIGIDO -----
16 // Erro original: funcao declarada como void, mas usa "return x + y;".
17 // Correcao: mudar o tipo de retorno para int (ou remover o return).
18 int printResults(int x, int y) {
19     cout << "A soma e " << x + y << '\n';
20     return x + y;
21 }
22
23 // ----- (c) CORRIGIDO -----
24 // Erro original: faltou a palavra-chave "class" (ou "typename") antes do A.
25 // Correcao: template <class A> ou template <typename A>.
26 template <class A>
27 A product(A num1, A num2, A num3) {
28     return num1 * num2 * num3;
29 }
30
31 // ----- (d) CORRIGIDO -----
32 // Erro original: nao se pode sobrecarregar funcoes que diferem APENAS
33 // no tipo de retorno. Para "corrigir", basta dar nomes diferentes
34 // (ou diferenciar tambem as listas de parametros).
35 double cubeD(int x) { return static_cast<double>(x) * x * x; }
36 int    cubeI(int x) { return x * x * x; }
37
38 int main() {
39     // Testes de (a)
40     cout << "sum<int>(1,2,3)          = " << sum(1, 2, 3) << endl;
41     cout << "sum<double>(1.5,2.5,3.0)= " << sum(1.5, 2.5, 3.0) << endl;
42
43     // Testes de (b)
44     int s = printResults(10, 20);
```

```

45     cout << "printResults retornou: " << s << endl;
46
47     // Testes de (c)
48     cout << "product<int>(2,3,4)      = " << product(2, 3, 4) << endl;
49     cout << "product<double>(1.5,2,4)= " << product(1.5, 2.0, 4.0) << endl;
50
51     // Testes de (d)
52     cout << "cubeD(3) = " << cubeD(3) << "   cubeI(4) = " << cubeI(4) << endl;
53
54     return 0;
55 }

```

Execução

Saída da execução

```

$ g++ -Wall -Wextra -std=c++14 ex15_10.cpp -o ex15_10
$ ./ex15_10
sum<int>(1,2,3)          = 6
sum<double>(1.5,2.5,3.0)= 7
A soma e 30
printResults retornou: 30
product<int>(2,3,4)      = 24
product<double>(1.5,2,4)= 12
cubeD(3) = 27   cubeI(4) = 64

```

Resumo — Tabela de diagnósticos

Item	Erro	Correção
(a)	Parâmetro de template não usado	Substituir <code>int</code> por <code>A</code> , ou remover <code>template</code>
(b)	<code>void</code> com <code>return x+y</code>	Mudar retorno para <code>int</code> , ou remover <code>return</code>
(c)	Falta <code>class</code> / <code>typename</code>	Inserir <code>class</code> ou <code>typename</code> antes de <code>A</code>
(d)	Sobrecarga apenas pelo retorno	Diferenciar nomes ou tipos de parâmetros

Observação de Engenharia de Software

Caça-aos-erros como o (d) acima ensinam um princípio importante do projeto de linguagens: *ambiguidade é o inimigo*. A escolha do C++ de *não* permitir sobrecarga apenas pelo tipo de retorno preserva a regra de que toda chamada de função tem uma e somente uma resolução — regra que torna possível ao compilador (e ao leitor humano) entender o código. Se esta regra fosse violada, todo `cube(5)` teria que ser examinado em seu contexto para decidir qual chamar.

Encerramento

Os Exercícios 15.5 a 15.10 oferecem o leitor um *primeiro contato prático* com C++: usamos `cin/cout` no lugar de `scanf/printf`, conhecemos `inline`, referências e templates, e identificamos as armadilhas mais comuns na transição a partir de C. Em todos os códigos, contudo, permanecemos no paradigma *procedural* — funções operando sobre dados.

A partir do Capítulo 16 daremos o passo conceitual fundamental: introduziremos a **classe** como mecanismo de encapsulamento de *dados* e *comportamento*. As funções deixarão de ser entidades soltas e tornar-se-ão *métodos* pertencentes a um tipo. O leitor que dominou tudo até aqui está perfeitamente preparado para isso.
